

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Лабораторная работа №1

(по дисциплине «Системы искусственного интеллекта»)

Выполнил:

Лабин Макар Андреевич,
СИИ 2.1, Р3331, 466449.

Проверил:

Запорожцев Иван Федорович,
кандидат технических наук



г. Санкт-Петербург, Россия
2026

Содержание

1	Задание	1
2	Выполнение работы	3
2.1	Описание задачи	3
2.1.1	Выбор датасета	3
2.1.2	EDA	3
2.2	Обучение моделей	7
2.2.1	Формулировка задачи	7
2.2.2	EDA	8
2.2.3	Обучение без регуляризации	9
2.2.4	Обучение с регуляризацией	11
2.3	Ответы на вопросы	14
2.4	Математическое обоснование MSE	14
3	Источники литературы	16

1 Задание

Task 1: [4 балла]

Выберите датасет и задачу для него (регрессия) — с Kaggle или [UCI Machine Learning Repository](#).

Рекомендации:

- количество признаков — не менее 10
- количество объектов — не менее 100
- возраст датасета — не старше 2010 года

Выполните EDA, а именно: придумайте вопросы к данным, которые потребуют использования следующих инструментов (напишите каждый вопрос и код к нему) [0,5 балла за покрытие каждого пункта]:

- группировка с агрегацией (groupby) (минимум два признака задействовано в пункте);
- изменение дискретности по времени (resample) (можно для этого пункта искусственно добавить время (поминутно → в час));
- объединение датафреймов (merge, join или concat) (можно искусственно разбить датафрейм на два: мальчики/девочки разбили по полу, потом собрали к исходному или другому);
- статическая визуализация seaborn;
- интерактивный график plotly;
- визуализация распределений (гистограмма относительных частот, kde, boxplot, violinplot, scatter);
- тесты на свойства распределения (например, нормальность);
- One Hot Encoding для категориальных признаков (если их нет, то придумайте как некие классы, определяемые по другим числовым признакам).

Task 2: [4 балла]

1. Выберите один из признаков как таргет (регрессант, предиктант), также выберите некоторое число признаков в качестве регрессоров (предикторов). Обоснуйте, что эти признаки имеют смысл использовать как регрессоры, исходя из предметной области (природы датасета), проведите аналитику — представьте код и рисунки. [1 балл]

Важно! Если выполняете исследование данных до разделения на обучающую, которая нужна для моделирования, и отложенную выборку (тестовую), то это потенциал для утечек (data leakage)! Следует сразу отделить часть данных, исследовать их только перед прогоном на уже обученной готовой модели!

2. Обучите линейную регрессионную модель без регуляризации. Проиллюстрируйте процесс обучения и тестирования модели, запишите выводы о качестве. Метрики: RMSE, MAE, R^2 [1 балл]

3. Обучите линейную регрессионную модель с регуляризацией (любой). Для предобработки осуществите удаление выбросов и нормирование — запишите обоснование для выбора способа поиска выбросов и нормирования. Используйте для этапов Pipeline из sklearn. Проиллюстрируйте процесс обучения и тестирования модели, запишите выводы о качестве. Метрики: RMSE, MAE, R^2 [2 балла]

Task 3: [2 балла — бонус]

По итогам лекции №1 ответьте на вопросы о месте искусственного интеллекта среди инноваций в крупной компании (пример «Норильского никеля» — см. видео в группе Telegram):

1. Какие задачи моделирования и управления решаются?
2. Какие разделы/формальные задачи ИИ задействованы?
3. Какие физико-химические признаки для них используются?
4. Укажите (предположите) для нескольких признаков: числовые или категориальные, временная и пространственная дискретность регистрации, сырой признак или агрегат по нескольким, сильно зашумленный, много ли пропусков, особенности закона распределения, потенциал для data leakage.

Task 4: [3 балла — бонус]

Покажите, что из максимизации правдоподобия выборки невязок, распределенных по нормальному закону, следует задача минимизации ошибки MSE, вычисленной по этим невязкам.

Обоснование принесет вам 2 балла.

Если выполните не на листе бумаги, а заполните ячейку (математические символы), то получите ещё 1 балл.

2 Выполнение работы

2.1 Описание задачи

2.1.1 Выбор датасета

Для рассмотрения задачи регрессии был выбран датасет Student Performance с репозитория UCI Machine Learning.^[1] Он имеет следующие характеристики:

- количество признаков — 30;
- количество объектов — 649;
- год публикации — 2014.

В датасете представлены результаты студентов среднего образования двух португальских школ. Данные включают оценки студентов, демографические, социальные и школьные характеристики, и были собраны с использованием школьных отчётов и анкет.

Предоставлены два набора данных, касающиеся успеваемости по двум различным предметам: Математика и Португальский язык.

Важное замечание от авторов: целевой признак G3 имеет сильную корреляцию с признаками G2 и G1. Это происходит потому, что G3 является итоговой оценкой за год (выданной в 3-м периоде), в то время как G1 и G2 соответствуют оценкам за 1-й и 2-й периоды.

2.1.2 EDA

Ниже приведены сформулированные вопросы и соответствующие решения.

```
1 import numpy as np
2
3 student_mat = pd.read_csv('/lab/student-mat.csv', sep=';')
4 student_por = pd.read_csv('/lab/student-por.csv', sep=';')
```

Листинг 1: Импорт данных для EDA.

Объединение датафреймов. Как объединить два датафрейма с информацией по двум разным предметам, чтобы получить один общий датафрейм?

```
1 student_gen = pd.concat([student_mat, student_por], ignore_index=True)
```

Группировка с агрегацией. Как различается средняя и медианная итоговая оценка (G3) студентов в зависимости от уровня образования матери (Medu) и пола (sex)?

```
1 groupby_aggr_result = student_gen.groupby(['Medu', 'sex'])['G3']
   .agg(['mean', 'median', 'count']).reset_index()
```

Листинг 2: Скрипт группировки с агрегацией.

	Medu	sex	mean	median	count
0	0	F	11.857143	11.0	7
1	0	M	13.000000	13.0	2
2	1	F	10.459677	11.0	124
3	1	M	9.730769	10.0	78
4	2	F	11.167598	12.0	179

Листинг 3: Вывод первых 5 строк `groupby_aggr_result`.

Изменение дискретности по времени. Если допустить, что в датасете есть информация о дате рождения студентов, как узнать среднюю итоговую оценку (G3) студентов по месяцам рождения?

```

1 from pandas import Timestamp, Timedelta
2
3 current_year = 2026
4 birth_years = current_year - data['age']
5 random_days = np.random.randint(0, 365, size=len(data))
6 data['bday'] = [
7     Timestamp(f'{year}-01-01') + Timedelta(days=int(day))
8     for year, day in zip(birth_years, random_days)
9 ]
10
11 resampled_df = (
12     data.resample('ME', on='bday')['G3']
13     .mean()
14     .reset_index(name='G3')
15 )

```

Листинг 4: Скрипт изменения дискретности по времени.

	bday	G3
0	2004-04-30	5.0
1	2004-05-31	NaN
2	2004-06-30	NaN
3	2004-07-31	NaN
4	2004-08-31	8.0

Листинг 5: Вывод первых 5 строк `resampled_df`.

Статическая визуализация seaborn. Каковы оценки студентов (G3) в зависимости от причины выбора школы (reason) с разбивкой по полу?

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 plt.figure(figsize=(8, 5))
5 sns.barplot(data=data, x='reason', y='G3', hue='sex', palette='
6     Set2')
7 plt.xlabel('Reason')
8 plt.ylabel('G3 Score')

```

```
plt.savefig(seaborn_plot_path)
```

Листинг 6: Скрипт визуализации с использованием `seaborn`.

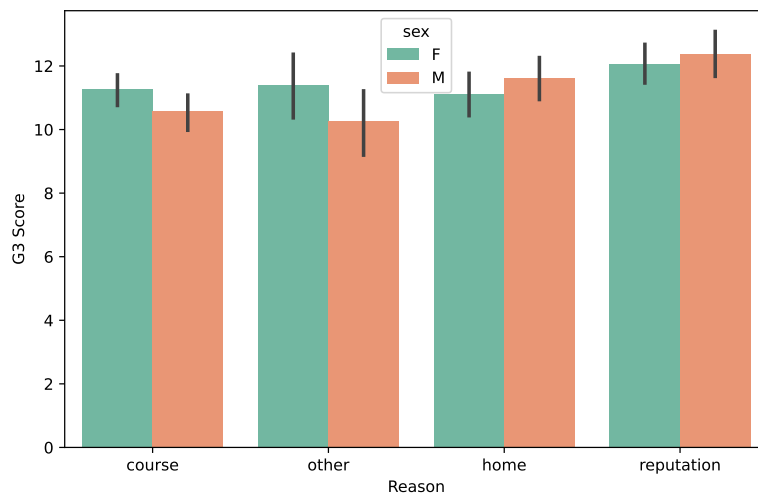


Рис. 1: Результат визуализации `seaborn`.

Визуализация распределений. Какова форма распределения оценок G3? Покажите это с помощью гистограммы относительных частот, плотности (KDE), ящика с усами (boxplot) и violinplot.

```
1 plt.figure(figsize=(8, 5))
2 sns.barplot(data=data, x='reason', y='G3', hue='sex', palette='
  Set2')
3 plt.xlabel('Reason')
4 plt.ylabel('G3 Score')
5 plt.savefig(seaborn_plot_path)
6 plt.close()
7
8 _, axes = plt.subplots(2, 2, figsize=(12, 10))
9
10 sns.histplot(data, y='G3', stat='probability', kde=True, ax=axes
11   [0, 0], color='skyblue')
12 axes[0, 0].set_title('Relative frequency histogram & KDE')
13
14 sns.boxplot(x=data['G3'], ax=axes[0, 1], color='lightgreen')
15 axes[0, 1].set_title('Boxplot of G3 scores')
16
17 sns.violinplot(
18   x=data['sex'],
19   y=data['G3'],
20   hue=data['sex'],
21   legend=False,
22   ax=axes[1, 0],
23   palette='pastel'
```

```

24 axes[1, 0].set_title('Violinplot of G3 scores by gender')
25
26 sns.scatterplot(data=data, x='age', y='G3', hue='sex', ax=axes
27                  [1, 1])
28 axes[1, 1].set_title('Scatterplot: age vs G3 score')
29
30 plt.tight_layout()
31 plt.savefig(plots_path)
32 plt.close()

```

Листинг 7: Скрипт визуализации распределений.

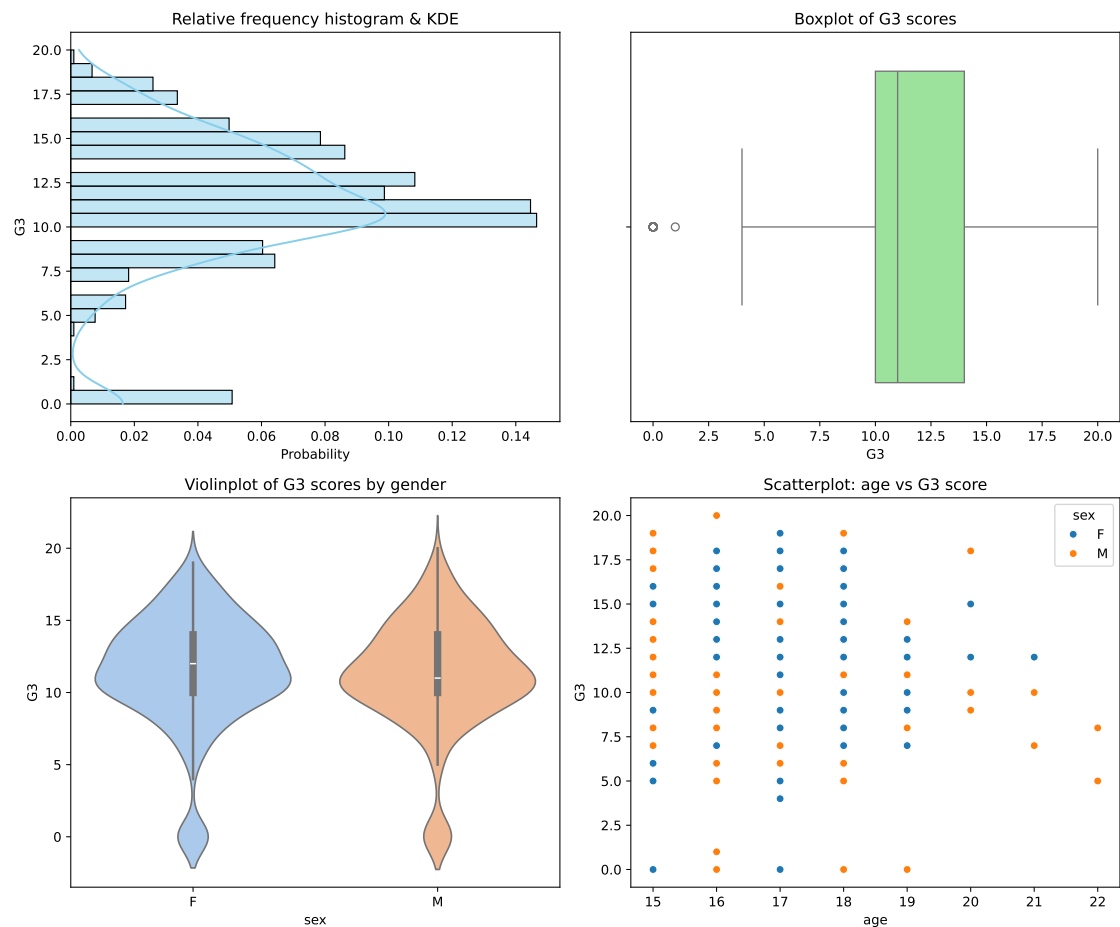


Рис. 2: Результат визуализации распределений.

One Hot Encoding. Как закодировать категориальный признак причины выбора школы (`reason`) в виде бинарных столбцов для использования в моделях машинного обучения?

```

1 reason_encoded = pd.get_dummies(data, columns=['reason'], prefix=
  'reason', drop_first=False)

```

Листинг 8: Скрипт One Hot Encoding.

```

1 reason_course reason_home reason_other reason_reputation

```

2	0	True	False	False	False
3	1	True	False	False	False
4	2	False	False	True	False
5	3	False	True	False	False
6	4	False	True	False	False

Листинг 9: Вывод первых 5 строк `reason_encoded`.

2.2 Обучение моделей

2.2.1 Формулировка задачи

Для оригинальности и с некоторой долей шутки возьмём в качестве целевой переменной (*регрессанта, предиктанта*) не итоговую оценку G3, как было сделано в оригинальной статье[2], а суммарное потребление алкоголя студентами. Его можно выразить как сумму двух признаков из датасета:

$$\text{TotalAlc} = \text{Dalc} + \text{Walc},$$

где:

- `Dalc` — потребление алкоголя в будние дни;
- `Walc` — потребление алкоголя в выходные дни.

Добавим новый признак `TotalAlc` в датафрейм:

```
1 df['TotalAlc'] = df['Dalc'] + df['Walc']
```

Листинг 10: Скрипт для создания целевой переменной `TotalAlc`.

В качестве фичей (*регрессоров, предикторов*) возьмём следующие признаки из датасета:

- `sex` (пол) — бинарный признак, который может влиять на социальные паттерны потребления алкоголя;
- `age` (возраст) — числовой признак, который может коррелировать с уровнем потребления алкоголя;
- `famsize` (размер семьи) — бинарный признак, который может отражать социальную среду студента;
- `Pstatus` (статус совместного проживания родителей) — бинарный признак, который может влиять на психологическое состояние студентов и их поведение;
- `Medu` (образование матери) — числовой признак, который может коррелировать с социальным статусом семьи и, следовательно, с поведением студентов;
- `Fedu` (образование отца) — числовой признак, аналогично `Medu`;

- **famrel** (качество семейных отношений) — числовой признак, который может отражать эмоциональную поддержку и влияние на поведение студентов;
- **goout** (проведение времени с друзьями) — числовой признак, который может коррелировать с социальными активностями студентов, включая потребление алкоголя;
- **absences** (количество пропусков) — числовой признак, который может отражать уровень ответственности и дисциплины студентов;
- **failures** (количество академических неудач) — числовой признак, который может быть связан с поведением студентов и их отношением к учебе.

2.2.2 EDA

Утечки данных. Учтём комментарий задания о том, что в задаче регрессии необходимо избегать утечки данных. Поэтому всё обучение моделей будет проводиться строго на обучающей выборке (**train**), а тестовая выборка (**test**) будет использоваться только для финальной оценки качества модели.

Параметры разбиения на обучающую и тестовую выборки:

- размер тестовой выборки (**test_size**) — 0.25;
- начальное значение генератора (**random_seed**) — 718234.

```

1 rng = np.random.default_rng(random_state)
2
3 shuffled_indices = rng.permutation(len(data))
4 test_set_size = int(len(data) * test_size)
5
6 test_indices = shuffled_indices[:test_set_size]
7 train_indices = shuffled_indices[test_set_size:]
8
9 train_data, test_data = data.iloc[train_indices], data.iloc[
    test_indices]
```

Листинг 11: Скрипт разбиения на обучающую и тестовую выборки.

Пустые значения. Проверим, что в выборках **train** и **test** нет пустых значений для признаков, которые будут использоваться в моделях:

```

1 target_column = 'TotalAlc'
2 categorical_columns = ['sex', 'famsize', 'Pstatus']
3 numeric_columns = ['age', 'Medu', 'Fedu', 'famrel', 'goout', '
    absences', 'failures', 'TotalAlc']
4 predictors = categorical_columns + numeric_columns
5 features = predictors + [target_column]
6
7 train_nans = train_data[features].isnull().sum().sum()
8 test_nans = test_data[features].isnull().sum().sum()
```

```

9
10 if train_nans > 0 or test_nans > 0:
11     raise ValueError(f"Found NaN values in train or test data.
        Train NaNs: {train_nans}, Test NaNs: {test_nans}")

```

Листинг 12: Скрипт для проверки на пустые значения в выборках.

В результате запуска скрипта выше ошибки не возникло, что подтверждает отсутствие пустых значений.

Категориальные признаки. Далее применим One Hot Encoding к категориальным признакам `sex`, `famsize` и `Pstatus` для подготовки данных к обучению моделей линейной регрессии:

```

1 data = pd.get_dummies(data, columns=categorical_columns,
        drop_first=False)

```

Листинг 13: Скрипт для One Hot Encoding категориальных признаков.

Нормализация признаков. В датасете присутствуют числовые признаки с разными диапазонами значений. Например, признак `age` может принимать значения от 15 до 22, в то время как признак `absences` может варьироваться от 0 до 93.

Для корректного обучения моделей линейной регрессии и интерпретации весов необходимо привести все числовые признаки к единому масштабу:

```

1 scaler = StandardScaler()
2 data[numerical_cols + [target_column]] = scaler.fit_transform(
        data[numerical_cols + [target_column]])

```

Листинг 14: Скрипт для нормализации числовых признаков.

Удаление выбросов. Для улучшения качества модели удалим выбросы из обучающей выборки на основе межквартильного размаха (IQR) для числовых признаков:

```

1 for col in numerical_columns:
2     Q1 = data[col].quantile(0.25)
3     Q3 = data[col].quantile(0.75)
4     IQR = Q3 - Q1
5     lower_bound = Q1 - 1.5 * IQR
6     upper_bound = Q3 + 1.5 * IQR
7     data = data[(data[col] >= lower_bound) & (data[col] <=
        upper_bound)]

```

Листинг 15: Скрипт для удаления выбросов из обучающей выборки.

2.2.3 Обучение без регуляризации

В качестве базовой модели выступает классическая линейная регрессия (метод наименьших квадратов, OLS), минимизирующая среднеквадратичную ошибку.

ку на обучающей выборке:

$$L_{OLS}(w) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \rightarrow \min_{\mathbf{w}}$$

```
1 from sklearn.linear_model import LinearRegression
2 from sklearn.metrics import root_mean_squared_error, r2_score
3
4 X_train = train_data.drop(columns=[target_column])
5 y_train = train_data[target_column]
6
7 X_test = test_data.drop(columns=[target_column])
8 y_test = test_data[target_column]
9
10 model = LinearRegression()
11 model.fit(X_train, y_train)
12
13 y_train_pred = model.predict(X_train)
14 y_test_pred = model.predict(X_test)
15
16 metrics = {
17     "train_rmse": root_mean_squared_error(y_train, y_train_pred)
18     ,
19     "test_rmse": root_mean_squared_error(y_test, y_test_pred),
20     "train_r2": r2_score(y_train, y_train_pred),
21     "test_r2": r2_score(y_test, y_test_pred),
22 }
23
24 _, axes = plt.subplots(1, 2, figsize=(12, 5))
25
26 axes[0].scatter(y_test, y_test_pred, alpha=0.6, color="blue")
27 axes[0].plot(
28     [y_test.min(), y_test.max()],
29     [y_test.min(), y_test.max()],
30     "r--",
31     lw=2,
32     label="Ideal Fit",
33 )
34 axes[0].set_xlabel("y_test")
35 axes[0].set_ylabel("y_pred")
36 axes[0].set_title("Fact vs Pred")
37 axes[0].legend()
38 axes[0].grid(True)
39
40 residuals = y_test - y_test_pred
41 axes[1].scatter(y_test_pred, residuals, alpha=0.6, color="purple")
42 axes[1].axhline(y=0, color="r", linestyle="--", lw=2)
43 axes[1].set_xlabel("y_pred")
44 axes[1].set_ylabel("y_test - y_pred")
45 axes[1].set_title("Residuals vs Predicted Values")
```

```

45 axes[1].grid(True)
46
47 plt.tight_layout()
48
49 plt.savefig(plot_path, dpi=300)
50 plt.close()

```

Листинг 16: Скрипт обучения классической линейной регрессии.

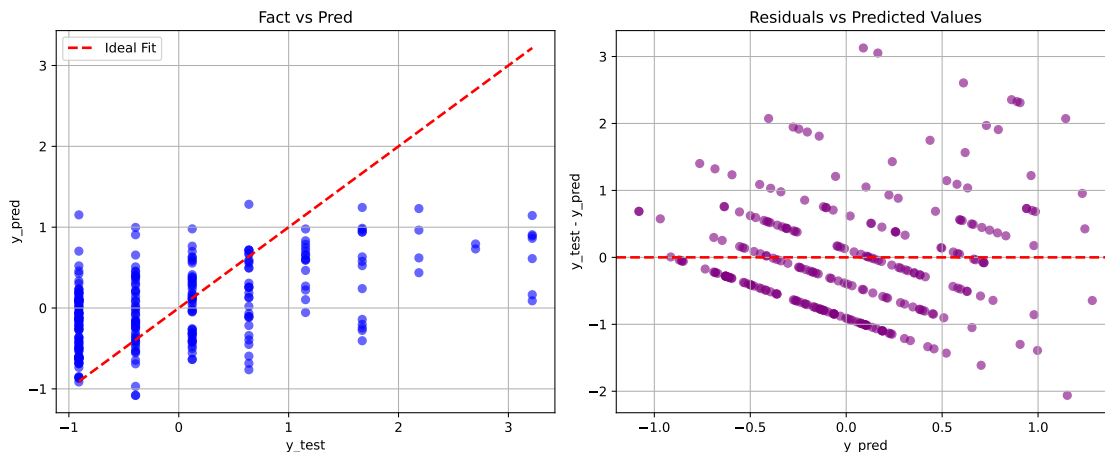


Рис. 3: Сравнение предсказанных и фактических значений TotalAlc на тестовой выборке.

Результаты оценки модели без регуляризации:

- $RMSE$ на обучающей выборке (**train**): 0.87695;
- $RMSE$ на тестовой выборке (**test**): 0.85817;
- Коэффициент детерминации (R^2) на обучающей выборке: 0.23096;
- Коэффициент детерминации (R^2) на тестовой выборке: 0.26355.

2.2.4 Обучение с регуляризацией

Для предотвращения переобучения применяются методы регуляризации.

Ridge-регрессия (L_2 -регуляризация). Добавляет штраф за сумму квадратов весов:

$$L_{Ridge}(w) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \alpha \sum_{j=1}^p w_j^2 \rightarrow \min_{\mathbf{w}}$$

Lasso-регрессия (L_1 -регуляризация). Добавляет штраф за сумму модулей весов, что способствует обнулению наименее важных коэффициентов и отбору признаков:

$$L_{Lasso}(w) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \alpha \sum_{j=1}^p |w_j| \rightarrow \min_{\mathbf{w}}$$

Мы воспользуемся L_1 регуляризацией для отбора признаков, а L_2 регуляризацией для снижения дисперсии модели.

```
1 from sklearn.linear_model import Ridge, Lasso
2
3 ridge_model = Ridge(alpha=alpha)
4 ridge_model.fit(X_train, y_train)
5
6 y_pred_ridge_train = ridge_model.predict(X_train)
7 y_pred_ridge_test = ridge_model.predict(X_test)
8
9 ridge_metrics = {
10     "train_rmse": root_mean_squared_error(y_train,
11         y_pred_ridge_train),
12     "test_rmse": root_mean_squared_error(y_test,
13         y_pred_ridge_test),
14     "train_r2": r2_score(y_train, y_pred_ridge_train),
15     "test_r2": r2_score(y_test, y_pred_ridge_test),
16 }
17
18 random_seed = context.get("random_seed", 42)
19 lasso_model = Lasso(alpha=alpha, random_state=random_seed)
20 lasso_model.fit(X_train, y_train)
21
22 y_pred_lasso_train = lasso_model.predict(X_train)
23 y_pred_lasso_test = lasso_model.predict(X_test)
24
25 lasso_metrics = {
26     "train_rmse": root_mean_squared_error(y_train,
27         y_pred_lasso_train),
28     "test_rmse": root_mean_squared_error(y_test,
29         y_pred_lasso_test),
30     "train_r2": r2_score(y_train, y_pred_lasso_train),
31     "test_r2": r2_score(y_test, y_pred_lasso_test),
32 }
33
34 _, axes = plt.subplots(1, 2, figsize=(14, 5))
35
36 coef_df = pd.DataFrame(
37     {
38         "Ridge (L2)": ridge_model.coef_.ravel(),
39         "Lasso (L1)": lasso_model.coef_.ravel(),
40     },
41     index=X_train.columns,
42 )
43
44 coef_df.plot(kind="bar", ax=axes[0], alpha=0.8)
45 axes[0].axhline(0, color="black", linestyle="--", linewidth=0.8)
46 axes[0].set_title("Feature Weights")
47 axes[0].set_ylabel("Weight")
48 axes[0].tick_params(axis="x", rotation=45)
49 axes[0].grid(True, linestyle=":", alpha=0.6)
```

```

46
47 axes[1].scatter(
48     y_test,
49     y_pred_lasso_test,
50     alpha=0.6,
51     color="blue",
52     label="Lasso",
53 )
54 axes[1].scatter(
55     y_test,
56     y_pred_lasso_test,
57     alpha=0.6,
58     color="orange",
59     label="Lasso",
60 )
61
62 y_min = float(y_test.min())
63 y_max = float(y_test.max())
64 axes[1].plot(
65     [y_min, y_max], [y_min, y_max], "r--", lw=2, label="Ideal
66     Fit"
67 )
68
69 axes[1].set_xlabel("y_test")
70 axes[1].set_ylabel("y_pred")
71 axes[1].set_title("Ridge vs Lasso: Fact vs Pred")
72 axes[1].legend()
73 axes[1].grid(True)
74
75 plt.tight_layout()
76 plt.savefig(plot_path, dpi=300)
77 plt.close()

```

Листинг 17: Скрипт обучения Ridge и Lasso регрессий с фиксированным гиперпараметром α .

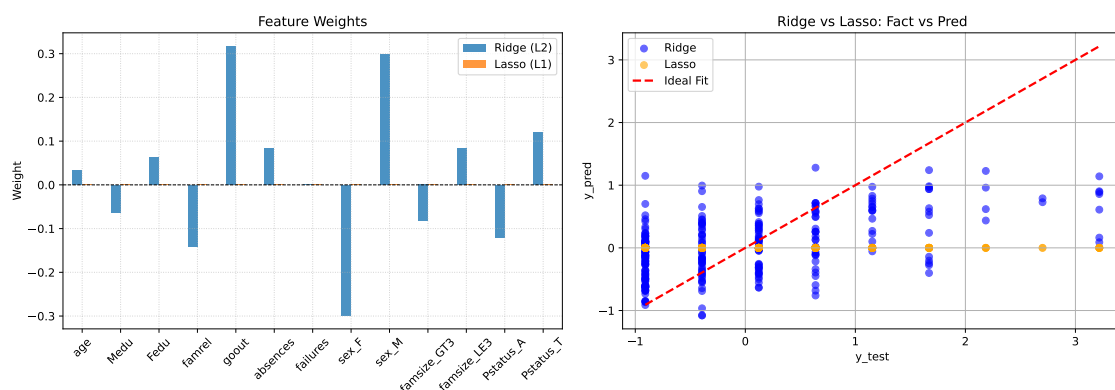


Рис. 4: Сравнение весов признаков и качества предсказаний для Ridge (L_2) и Lasso (L_1) регрессий при $\alpha = 1$.

Результаты моделей с регуляризацией на тестовой выборке:

- **Ridge (L_2):** $RMSE = 0.85814$, $R^2 = 0.2636$;
- **Lasso (L_1):** $RMSE = 1.0$, $R^2 = 0.0$.

2.3 Ответы на вопросы

...

2.4 Математическое обоснование MSE

Покажем, что из максимизации функции правдоподобия (Maximum Likelihood Estimation, MLE) выборки невязок, распределенных по нормальному закону, строго следует задача минимизации среднеквадратичной ошибки (MSE).

1. Формализация задачи и предпосылки Рассмотрим модель линейной регрессии:

$$y_i = \mathbf{w}^T \mathbf{x}_i + \varepsilon_i, \quad i = 1, \dots, N \quad (1)$$

где y_i — целевая переменная, $\mathbf{x}_i$ — вектор признаков i -го объекта, $\mathbf{w}$ — вектор параметров модели, а $\varepsilon_i = y_i - \mathbf{w}^T \mathbf{x}_i$ — невязка (ошибка) модели.

Основное предположение: Невязки ε_i являются независимыми и одинаково распределенными (*i.i.d.*) случайными величинами, подчиняющимися нормальному закону с нулевым математическим ожиданием и постоянной дисперсией σ^2 :

$$\varepsilon_i \sim \mathcal{N}(0, \sigma^2) \quad (2)$$

Плотность вероятности для отдельной невязки ε_i имеет вид:

$$p(\varepsilon_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\varepsilon_i^2}{2\sigma^2}\right) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mathbf{w}^T \mathbf{x}_i)^2}{2\sigma^2}\right) \quad (3)$$

2. Построение функции правдоподобия (Likelihood Function) В силу независимости выборки невязок, совместная плотность вероятности (функция правдоподобия $L(\mathbf{w}, \sigma^2)$) равна произведению плотностей отдельных наблюдений:

$$L(\mathbf{w}, \sigma^2) = \prod_{i=1}^N p(\varepsilon_i) = \prod_{i=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mathbf{w}^T \mathbf{x}_i)^2}{2\sigma^2}\right) \quad (4)$$

Объединяя множители, получаем:

$$L(\mathbf{w}, \sigma^2) = (2\pi\sigma^2)^{-\frac{N}{2}} \exp\left(-\frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2\right) \quad (5)$$

3. Переход к логарифмической функции правдоподобия (Log-Likelihood)

Задача максимизации правдоподобия по параметрам $\mathbf{w}$ имеет вид:

$$\max_{\mathbf{w}} L(\mathbf{w}, \sigma^2) \quad (6)$$

Поскольку строго возрастающая функция $\ln(x)$ сохраняет точки экстремума, перейдем к логарифму функции правдоподобия $\ell(\mathbf{w}, \sigma^2) = \ln L(\mathbf{w}, \sigma^2)$:

$$\ell(\mathbf{w}, \sigma^2) = \ln \left((2\pi\sigma^2)^{-\frac{N}{2}} \exp \left(-\frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \right) \right) \quad (7)$$

$$= -\frac{N}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \quad (8)$$

4. Оптимизация по параметрам модели $\mathbf{w}$ Найдем оценку максимального правдоподобия $\mathbf{w}_{\text{MLE}}$:

$$\mathbf{w}_{\text{MLE}} = \arg \max_{\mathbf{w}} \left(-\frac{N}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \right) \quad (9)$$

Преобразуем оптимизируемое выражение:

1. Отбросим слагаемое $-\frac{N}{2} \ln(2\pi\sigma^2)$, так как оно не зависит от $\mathbf{w}$:

$$\mathbf{w}_{\text{MLE}} = \arg \max_{\mathbf{w}} \left(-\frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \right) \quad (10)$$

2. Опустим положительную константу $\frac{1}{2\sigma^2} > 0$:

$$\mathbf{w}_{\text{MLE}} = \arg \max_{\mathbf{w}} \left(-\sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \right) \quad (11)$$

3. Заменяем знак функции (умножение на -1), преобразуя задачу максимизации в эквивалентную задачу **минимизации**:

$$\mathbf{w}_{\text{MLE}} = \arg \min_{\mathbf{w}} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \quad (12)$$

5. Переход к метрике MSE Умножение на положительный масштабирующий коэффициент $\frac{1}{N}$ не меняет точку минимума:

$$\mathbf{w}_{\text{MLE}} = \arg \min_{\mathbf{w}} \left(\frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \right) \quad (13)$$

По определению, среднеквадратичная ошибка $\text{MSE}(\mathbf{w})$ на выборке равна:

$$\text{MSE}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \varepsilon_i^2 = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \quad (14)$$

Таким образом:

$$\mathbf{w}_{\text{MLE}} = \arg \min_{\mathbf{w}} \text{MSE}(\mathbf{w}) \quad (15)$$

Q.E.D.

3 Источники литературы

1. *Cortez P.* Student Performance Dataset / UCI Machine Learning Repository. — 2014. — URL: <https://archive.ics.uci.edu/dataset/320/student+performance> (дата обр. 24.09.2026).
2. *Cortez P., Silva A. M. G.* Using Data Mining to Predict Secondary School Student Performance // A. Brito and J. Teixeira Eds., Proceedings of 5th FUTURE BUSINESS TECHNOLOGY Conference (FUBUTEC 2008). — 2008. — С. 5—12. — URL: https://www.researchgate.net/publication/228780408_Using_data_mining_to_predict_secondary_school_student_performance (дата обр. 24.09.2026).